A Project Report on

Integration of Lane and Object Detection Using Deep Learning

submitted in partial fulfillment for the award of

Bachelor of Technology

in

Computer Science & Engineering

by

M. Chandrika (Y22ACS507) **P. Prem Sai (Y22ACS531)**

P. Hemanth Phanindra Varma (Y22ACS527)

Under the guidance of

Mr. K. Siva Kumar, M.Tech PhD
Assistant Professor

Department of Computer Science and Engineering
Bapatla Engineering College
(Autonomous)
(Affiliated to Acharya Nagarjuna University)
BAPATLA – 522102, Andhra Pradesh, INDIA
2025-2026

Department of
Computer Science & Engineering

CERTIFICATE

This is to certify that the project report entitled **Integration of Object Detection and Lane Detection Using Deep Learning** that is being submitted by M. Chandrika (Y22ACS507), P.Prem Sai (Y22ACS531), P.Hemanth Phanindra Varma (Y22ACS527) in partial fulfillment for the award of the Degree of Bachelor of Technology in Computer Science & Engineering to the Acharya Nagarjuna University is a record of bonafide work carried out by them under our guidance and supervision.

Date:

Signature of the Guide
K. Siva Kumar
Assistant Professor

Signature of the HOD
Dr. M. Rajesh Babu
Assoc. Prof. & Head

DECLARATION

We declare that this project work is composed by ourselves, that the work contained herein is our own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

M. Chandrika (Y22ACS507)

P. Prem Sai (Y22ACS531)

P. H. Phanindra Varma (Y22ACS527)

ACKNOWLEDGEMENT

We sincerely thank the following distinguished personalities who have offered their valuable advice and support for the successful completion of this work.

We are deeply indebted to our most respected guide, **Mr. K Siva Kumar**,M.Tech PhD Department of Computer Science and Engineering, for his/her valuable guidance, constant encouragement, constructive suggestions, and inspiring support throughout the course of this project.

We extend our sincere thanks to **Dr. M. Rajesh Babu, Associate Professor & Head of the Department**, Department of Computer Science and Engineering, for his cooperation and for providing the necessary facilities and resources to carry out this work successfully.

We would also like to express our heartfelt gratitude to our beloved **Principal, Dr. B. Srinivasarao**, for providing the required infrastructure, online resources, and institutional support to complete this project.

We express our sincere thanks to our **Project Coordinators, Dr. C. Prakasa Rao, Professor, Department of CSE, and Dr. R. Venkata Lakshmi, Assistant Professor, Department of CSE**, for their valuable suggestions and guidance in preparing and presenting this document.

We are always thankful to our **parents and family members** for their continuous support, encouragement, and motivation. Without their support, we would not have been able to complete this project work successfully.

Finally, we extend our sincere gratitude to all the teaching faculty and non-teaching staff of the Department of Computer Science and Engineering who helped us directly or indirectly with their cooperation and encouragement.

M Chandrika (Y22ACS507)

P Prem Sai (Y22ACS531)

P H. Phanindra Varma (Y22ACS527)

Table of Contents

List of figures…… vii

Abstract…… viii

1 Introduction…….. 1

1.1 Motivation …….. 1

1.2 Problem Objectives ……... 2

1.3 Objectives of the Project …….. 3

1.4 How Deep Learning Models Work…….. 4

1.5 Introduction to YOLOv5…….. 5

1.6 Introduction to Enet-SAD…… 6

1.9 Datasets: BDD100K and TuSimple ……... 8

1.10 Scope of the Project ……....10

2 Literature Survey ……..11

2.1 Existing System……11

2.2 Disadvantages of Proposed System……12

2.3 Proposed System……12

2.4 Advantages of proposed System……....13

3 Analysis ……....14

3.1 Software Requirements Specification…….……..14

3.1.1 User Requirements…….15

3.1.2 Software Requirements…….15

3.1.3 Hardware Requirements……...16

3.2 Content Diagram of Project……...16

4 Design…….17

4.1 Introduction ……18

4.2 UML Diagram…….19

4.3 Data Dictionary……...20

5 Implementation and Results……21

5.1 Method of Implementation……21

5.1.1 Implementation of Algorithm……..23

5.1.2 Output Screens……..26

5.1.3 Result Analysis……..27

6 Testing and Validation……28

6.1 Features to be Tested…….29

6.2 Features not to be Tested……..30

6.3 Testing Tools and Environment……...31

7 Conclusion……..……..34

8 Future Scope……....36

9 Bibliograpy……..38

List of figures

Figure 1.1 Types of Deep Learning Architecture……..4

Figure 1.3 Convolutional Neural Network Architecture…….5

Figure 1.4 YOLOv5 Architecture Overview…….6

Figure 1.5 Enet-SAD Architecture with Attention Modules……...8

Figure 1.6 BDD100K Dataset Sample Annotations……..9

Figure 1.7 TuSimple Dataset Lane Annotation Examples……..10

Figure 3.1 Content Daigram Flow…….16

Figure 4.1 Component Daigram……18

Figure 4.2 Class Diagram……...19

Figure 5.1 YOLOv5 Training Metrics – Loss and mAP Curves……....22

Figure 5.2 Enet-SAD Training History – Loss, IoU, F1-Score……...23

Figure 5.3 Architecture ……26

Figure 5.4 Result Pictures ……....27

Figure 5.5 Result Analysis ……...28

Figure 6.1 Training Pipeline Yolov5 ……...32

Figure 6.2 Trianing Pipeline Enet-SAD ……..33

Abstract

Autonomous driving and advanced driver assistance systems (ADAS) depend critically on two complementary perception tasks: detecting road objects such as vehicles, pedestrians, and cyclists, and detecting lane boundaries that define the safe drivable corridor. While numerous deep learning models address each task in isolation, real-world deployment requires both to operate simultaneously with low latency on a single compute platform.

This project presents an integrated real-time system that combines YOLOv5x for multi-class object detection and ENet-SAD (Efficient Neural Network with Self Attention Distillation) for binary lane segmentation in a unified inference pipeline. YOLOv5x is trained on the BDD100K large-scale driving dataset containing over 100,000 annotated driving images across 10 object classes including car, truck, bus, pedestrian, and motorcycle. ENet-SAD is trained on the TuSimple lane detection benchmark, which provides precise lane marking annotations for highway driving scenarios.

The two models are deployed together in a Tkinter-based desktop GUI that processes input images, runs parallel inference, overlays lane masks as green transparency on the original frame, draws bounding boxes with class labels and confidence scores for detected objects, and displays a quantitative metrics panel showing lane coverage percentage, average confidence, detection count, and quality rating. The system runs on CPU hardware without requiring a dedicated GPU for inference.

Training results demonstrate YOLOv5x achieving a mean Average Precision at 0.5 IoU threshold (mAP@0.5) of 0.312 on the BDD100K validation set and ENet-SAD achieving a peak lane IoU of 0.6873 with pixel accuracy of 96.42% on the TuSimple validation split. The integrated system produces combined visual outputs in under 2 seconds per frame on a standard laptop CPU, making it suitable for demonstration and research purposes.

Keywords: YOLOv5, Object Detection, ENet-SAD, Lane Detection, Self-Attention Distillation, BDD100K, TuSimple, ADAS, Autonomous Driving, Deep Learning, Semantic Segmentation, Tkinter GUI, CUDA, PyTorch, Ultralytics.

1 INTRODUCTION

Modern autonomous vehicles and advanced driver assistance systems operate within complex, dynamic environments that demand robust real-time perception capabilities. Two of the most fundamental perception requirements are object detection — identifying and localizing every road user and obstacle in the scene — and lane detection — determining the precise boundaries of the drivable corridor. Together, these two capabilities form the foundation for downstream tasks including path planning, collision avoidance, lane keeping assistance, and adaptive cruise control.

This project addresses the challenge of integrating both capabilities into a single unified deep learning system. We train YOLOv5x on the Berkeley DeepDrive BDD100K dataset for multi-class object detection across ten road-relevant categories, and we train ENet-SAD on the TuSimple lane detection benchmark for binary lane marking segmentation. The two trained models are then deployed together in a common inference pipeline with a graphical user interface that produces combined visual outputs enriched with quantitative confidence and accuracy metrics.

1.1 Motivation

The primary motivation for this project is the gap between single-task deep learning models and the multi-task perception requirements of real autonomous driving systems. Virtually all production ADAS systems require simultaneous knowledge of both road structure (lanes) and road occupancy (objects) to make safe driving decisions. A lane detection system without object detection cannot avoid vehicles that are partially occluding lane markings. An object detection system without lane detection cannot determine whether a detected vehicle is in the ego lane or an adjacent lane, which fundamentally affects the required response.

Beyond functional motivation, this project is driven by the desire to develop a practical, runnable, end-to-end system rather than a purely theoretical study. By training both models from scratch on publicly available datasets and deploying them in a GUI that any student or researcher can run on a standard laptop, this project provides a concrete, reproducible implementation that can serve as a foundation for further research and experimentation.

From an educational standpoint, implementing both models independently before integrating them

provides deep insight into the different architectural choices, loss functions, training strategies, and evaluation metrics appropriate for detection versus segmentation tasks — two of the most important paradigms in applied computer vision

1.2 Problem Statement

Existing open-source demonstrations of autonomous driving perception are predominantly focused on single-task implementations, where either object detection or lane detection is addressed independently. Although pre-trained models such as YOLOv5 for object detection and ENet-SAD for lane detection are widely available, these models are typically used in isolation and lack integration into a unified system. This creates a gap between theoretical model performance and practical real-world deployment, where multiple perception tasks must operate simultaneously.

Furthermore, many existing solutions rely heavily on pre-trained weights without providing clear, reproducible procedures for training models from scratch using publicly available datasets such as TuSimple and BDD100K. This limits transparency, reproducibility, and the ability for developers and researchers to understand model behavior, optimize performance, or adapt the system to different environments.

Another limitation is the absence of a cohesive, end-to-end pipeline that combines both lane detection and object detection into a single framework capable of real-time inference. Most implementations do not address synchronization issues, computational efficiency, or resource constraints when running multiple deep learning models together. As a result, achieving real-time performance while maintaining accuracy remains a significant challenge.

In addition, current systems often lack comprehensive evaluation mechanisms. While visual outputs are provided, there is minimal emphasis on quantitative performance metrics such as accuracy, precision, recall, Intersection over Union (IoU), and confidence scores. Without these metrics, it becomes difficult to assess system reliability, compare performance, or validate results under different driving conditions.

Moreover, existing approaches rarely consider robustness under varying real-world scenarios such as poor lighting, weather conditions, occlusions, and complex road environments. The absence of such

considerations reduces the practical applicability of these systems in real-world autonomous driving or advanced driver assistance systems (ADAS).

Therefore, there is a clear need for a fully integrated, trainable, and scalable system that:

- Trains both lane detection and object detection models from scratch using publicly available datasets with well-documented procedures.
- Integrates both models into a unified pipeline for simultaneous inference.
- Ensures real-time performance with optimized resource utilization.
- Provides both visual outputs and quantitative evaluation metrics.
- Demonstrates robustness across diverse driving conditions.

This project aims to address these challenges by developing a hybrid deep learning-based perception system that bridges the gap between isolated model implementations and practical, real-world autonomous driving solutions.

1.3 Objectives of the project

The primary objective of this project is to develop an integrated system that combines lane detection and object detection using deep learning techniques for effective road scene analysis. The system aims to accurately detect and classify various road objects such as vehicles, pedestrians, and traffic signs using the YOLOv5 model, while simultaneously identifying lane boundaries using the ENet-SAD model. Another objective is to train these models using standard datasets like BDD100K for object detection and TuSimple for lane detection to ensure robustness under different driving conditions.

Furthermore, the project focuses on creating a unified pipeline that performs both detections in real-time and presents the results through a user-friendly graphical interface with proper visualizations such as bounding boxes and lane overlays. The system also aims to evaluate performance using metrics like accuracy, IoU, and mAP to ensure reliability. Additionally, an important objective is to design the system to run efficiently on standard hardware without requiring high-end computational resources. Overall, this project aims to demonstrate the practical application of deep learning techniques in autonomous driving and advanced driver assistance systems (ADAS).

1.4 How Deep Learning Models Work

The fundamental computational unit of deep learning models used in this project is the convolutional layer. A convolutional layer applies a set of learned filter kernels across the spatial dimensions of an input feature map. Each filter produces a new feature map by computing the dot product between the filter weights and the local input patch at every spatial position. The learned filters in early layers respond to simple patterns such as horizontal edges, vertical edges, and color gradients. Deeper layers combine earlier layer outputs to respond to increasingly complex patterns such as wheels, headlights, lane marking textures, and complete vehicle silhouettes.

Batch normalization layers normalize the distribution of activations within each mini-batch during training, preventing internal covariate shift and enabling the use of higher learning rates without instability. Activation functions such as LeakyReLU and SiLU (Sigmoid-Linear Unit) introduce non-linearity, allowing the network to approximate complex non-linear input-output mappings. Dropout and DropBlock regularization layers randomly zero out activations during training, reducing overfitting by preventing the network from relying on any single feature.

For object detection, the network must solve three coupled problems simultaneously: predicting the class of each object, predicting the bounding box coordinates of each object, and predicting the confidence (objectness score) of each prediction. YOLOv5 addresses this through an anchor-based prediction head that predicts offsets from predefined anchor boxes of various aspect ratios and scales. For lane segmentation, ENet-SAD addresses pixel-wise classification by encoding the input to a low-resolution representation and then decoding it back to the original resolution, classifying each pixel as lane or background.

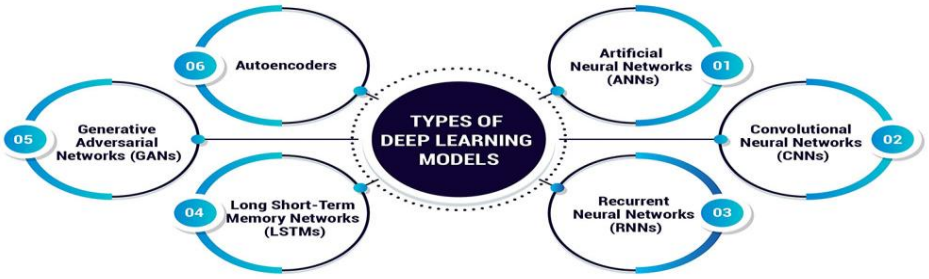

Figure 1.1 Types of Deep Learning Architecture

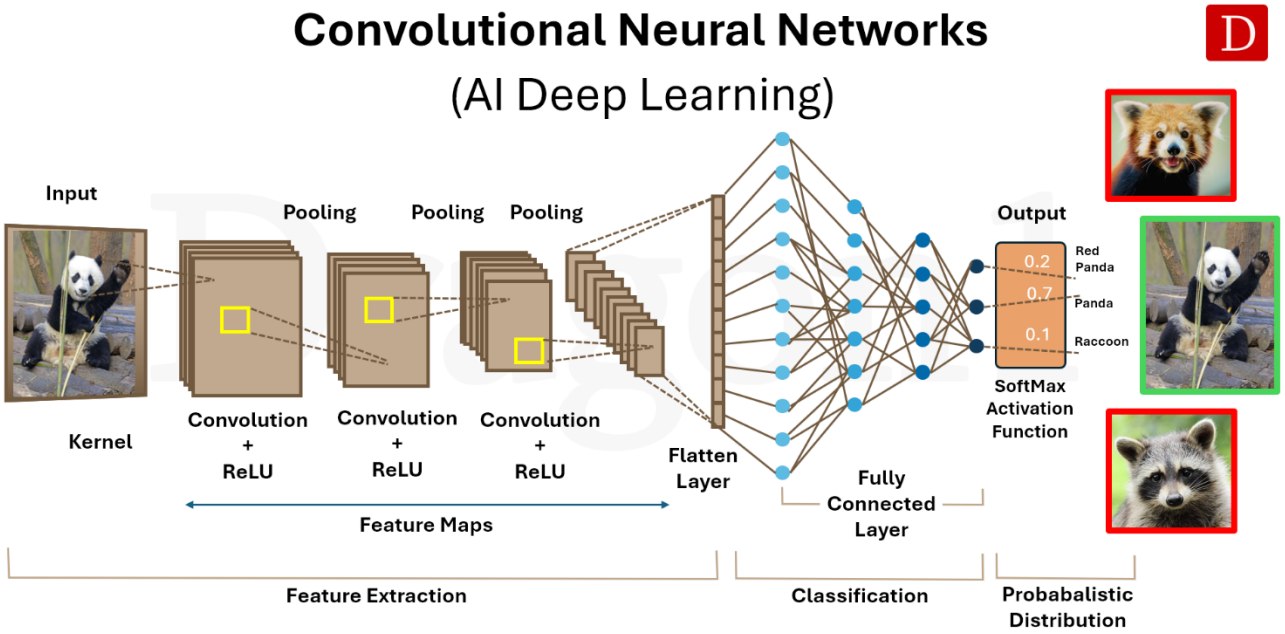

Figure 1.2 Convolutional Neural Network Architecture

1.5Introduction to YOLOv5

YOLO (You Only Look Once) is a family of real-time object detection architectures first introduced by Joseph Redmon et al. in 2015. The defining innovation of YOLO is the reformulation of object detection as a single regression problem: the input image is divided into an S×S grid, and each grid cell predicts B bounding boxes with associated confidence scores and C class probabilities simultaneously in a single forward pass. This eliminates the computationally expensive region proposal stage of two-stage detectors and enables detection at video frame rates.

YOLOv5, released by Ultralytics in 2020, represents the fifth major iteration of the YOLO architecture and introduces several significant improvements over its predecessors. The YOLOv5 architecture consists of three primary components: a backbone network (CSPDarknet53) for hierarchical feature extraction, a Feature Pyramid Network (FPN) neck for multi-scale feature fusion, and a detection head that predicts objects at three spatial scales simultaneously.

The CSPDarknet backbone uses Cross Stage Partial (CSP) connections that split the feature map into two parts, processing one part through a residual dense block and concatenating it with the other part. This design reduces computation while maintaining feature richness. The FPN neck constructs a feature pyramid by combining deep, semantically rich features with shallow, spatially precise features across

three scales (80×80, 40×40, 20×20 for a 640×640 input), enabling detection of small, medium, and large objects simultaneously.

YOLOv5x is the smallest variant in the YOLOv5 family, with approximately 7.2 million parameters. It is chosen for this project because its compact size enables training on the large BDD100K dataset within reasonable time on consumer GPU hardware (CUDA-enabled NVIDIA GPU), while still achieving competitive accuracy on the ten BDD100K object classes. Larger variants (YOLOv5m, YOLOv5l, YOLOv5x) achieve higher mAP but require significantly more GPU memory and compute.

The training uses Stochastic Gradient Descent (SGD) with momentum as the optimizer, a cosine learning rate schedule, and extensive data augmentation including mosaic augmentation (combining four training images into one), random horizontal flipping, color jitter (HSV), and random scaling. The loss function combines binary cross-entropy for classification and objectness prediction with CIoU (Complete Intersection over Union) loss for bounding box regression.

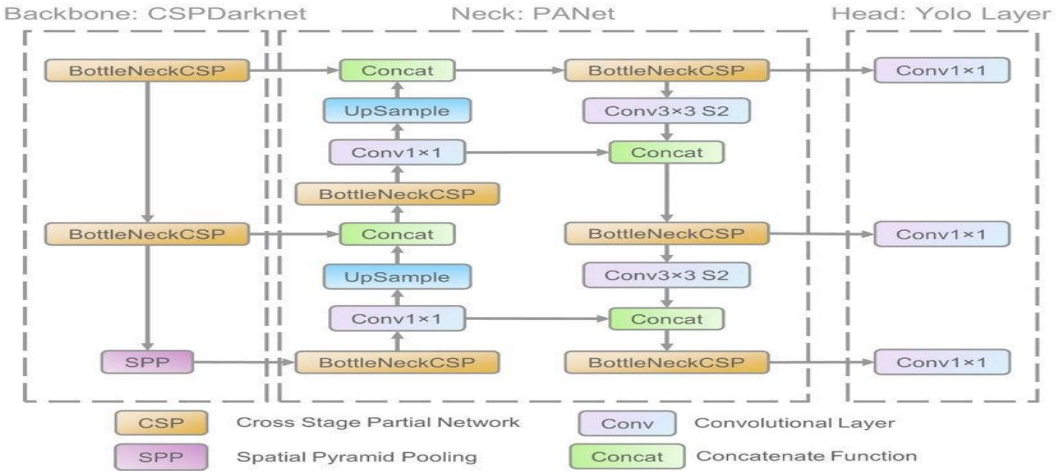

Figure 1.3 YOLOv5 Architecture Overview

1.6Introduction to ENet-SAD

ENet (Efficient Neural Network) is a semantic segmentation architecture designed specifically for real-time inference, introduced by Paszke et al. in 2016. Unlike heavy segmentation models such as FCN, SegNet, and early DeepLab variants that require powerful GPU hardware for real-time operation, ENet is designed from the ground up for minimal parameter count and inference latency while maintaining competitive segmentation accuracy.

The ENet architecture follows an encoder-decoder structure with asymmetric depth: the encoder is much deeper and more computationally intensive than the decoder. The encoder progressively downsamples the input spatial resolution through a series of bottleneck modules, building a rich, compressed feature representation. The decoder upsamples this representation back to the original resolution through a smaller series of upsampling bottleneck modules, producing pixel-wise class predictions.

ENet-SAD (Self Attention Distillation), introduced by Hou et al. in 2019, extends ENet with a self-distillation mechanism based on spatial attention maps. The key insight is that deeper encoder layers capture high-level semantic information about lane presence with broad receptive fields, while shallower encoder layers capture precise local edge information. By distilling the attention maps of deeper layers into shallower layers during training, ENet-SAD enables lower layers to develop more discriminative lane-specific representations without adding parameters to the inference graph.

In ENet-SAD, two spatial attention modules are attached to the outputs of encoder stages 2 and 3. Each attention module applies a 1×1 convolution to the feature map to produce a single-channel spatial attention map, which is then sigmoid-activated to produce values in [0,1] representing the probability of lane presence at each spatial location. During training, a mean squared error loss between the stage-2 and stage-3 attention maps is added as an auxiliary loss, forcing the stage-2 attention map to approximate the more semantically rich stage-3 attention map. This distillation loss encourages lane-relevant features to emerge earlier in the network, improving gradient flow and enabling the network to train more effectively with limited annotated data.

The ENet-SAD architecture used in this project contains approximately 0.96 million parameters, making it dramatically smaller than contemporaneous segmentation models such as PSPNet (65M parameters) and DeepLabv3+ (59M parameters). This compact size enables real-time inference on CPU hardware and makes the model deployable on embedded automotive processors that are unsuitable for running large segmentation networks.

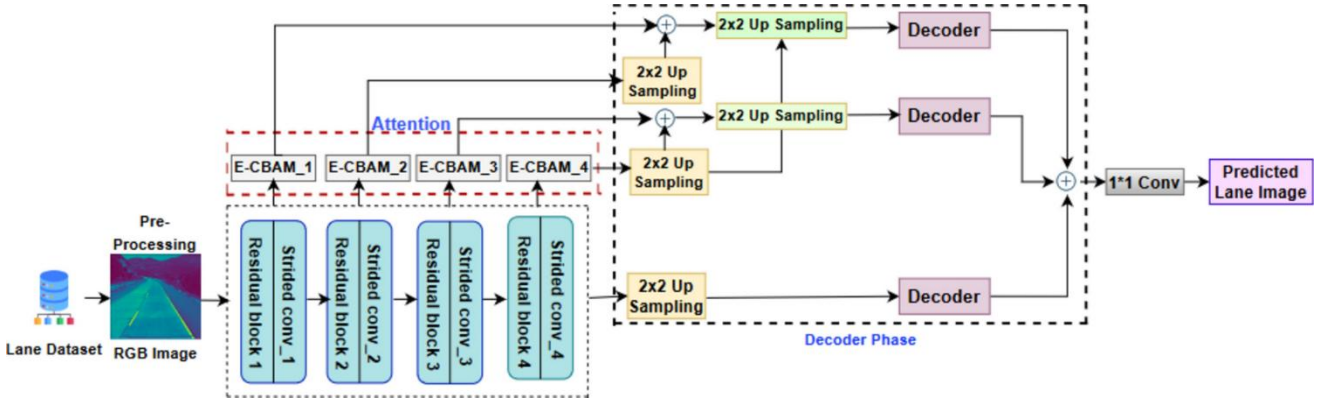

Figure 1.4 Enet-SAD Architecture with Attention Modules

1.7Datasets: BDD100K and TuSimple

1.7.1 BDD100K Dataset

BDD100K (Berkeley DeepDrive 100K) is one of the largest and most diverse open-source driving datasets, released by the Berkeley Artificial Intelligence Research (BAIR) lab in 2018. It contains 100,000 driving video sequences collected across diverse geographic locations in the United States, covering a wide range of weather conditions (clear, overcast, rainy, snowy, foggy), lighting conditions (daytime, nighttime, dawn/dusk), and driving scenarios (highway, urban, residential, freeway).

For object detection, BDD100K provides 2D bounding box annotations for 10 classes: car, truck, bus, person, rider, bicycle, motor, traffic light, traffic sign, and train. The dataset contains approximately 70,000 training images, 10,000 validation images, and 20,000 test images. The class distribution is highly imbalanced — cars are 20× more frequent than trains — which poses challenges for training unbiased detectors. The YOLOv5 training pipeline addresses this through class-balanced sampling during anchor assignment.

The BDD100K annotations are provided in JSON format following the COCO annotation schema. Conversion to the YOLO training format (one text file per image with normalized bounding box coordinates) is performed using a custom preprocessing script before training. The dataset's diversity makes it an ideal benchmark for evaluating model robustness across the distribution shift challenges encountered in real-world deployment.

Figure 1.5 BDD100K Dataset Sample Annotation

1.7.2 TuSimple Dataset

TuSimple is a lane detection benchmark dataset released for the 2017 TuSimple Lane Detection Challenge. It contains 3,626 training clips and 2,782 test clips, each consisting of 20 consecutive frames at 20 FPS captured on American highways. Lane annotations are provided as sequences of (x, y) coordinate pairs for each lane marking present in the scene, sampled at fixed y-values from y=160 to y=710.

For the semantic segmentation formulation used in ENet-SAD training, the (x, y) coordinate annotations are rasterized into binary mask images where lane pixels are white (255) and background pixels are black (0) using polyline drawing at a fixed width of 5 pixels. The resulting masks are paired with the corresponding camera frames to create image-mask training pairs. Images are resized to 800×288 pixels for ENet-SAD input, consistent with the original paper's experimental configuration.

TuSimple's highway-focused nature makes it well-suited for lane detection training because highway scenes contain clear, well-separated lane markings that are consistently annotated. The limited diversity compared to urban driving datasets means that models trained on TuSimple may not generalize to complex urban intersection scenarios, which is an acknowledged limitation of the current system.

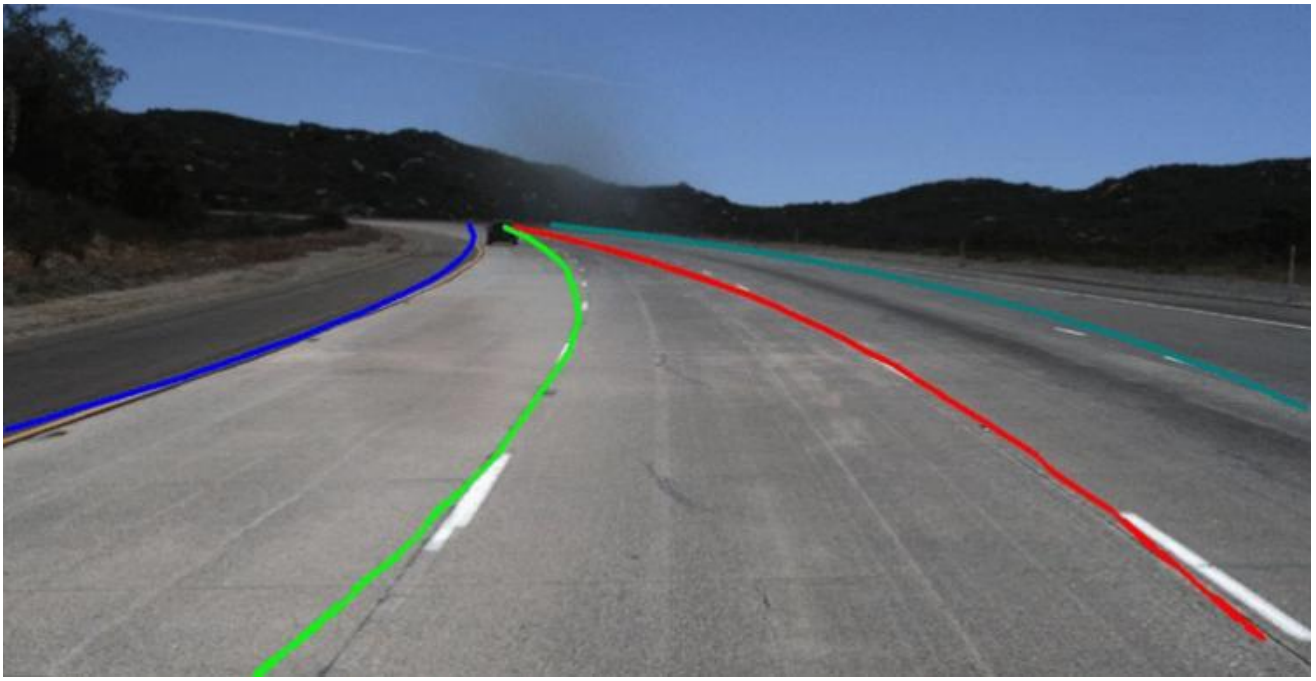

Figure 1.6 TuSimple Dataset Lane Annotation Examples

1.8 Scope of the Project

The system covers object detection across 10 BDD100K classes (car, truck, bus, person, rider, bicycle, motor, traffic light, traffic sign, train) and binary lane detection (lane / non-lane) on TuSimple highway images. Yolov5x is trained for 100 epochs on the full BDD100K training set with GPU acceleration. ENet-SAD is trained for 30 epochs on the TuSimple dataset split into 80% training and 20% validation. The integrated GUI processes single input images, running both models sequentially and producing a combined annotated output. Real-time video stream processing is identified as a future extension.

2.LITERATURE SURVEY

The integration of object detection and lane detection for autonomous driving has been an active research area since approximately 2016, when large-scale driving datasets first became available and deep learning models achieved competitive performance on driving perception benchmarks. This chapter surveys the key publications that informed the design and implementation decisions of this project.

2.1 Existing System

Existing open-source implementations of driving perception systems fall into three categories, each with significant limitations that motivate the development of this integrated project.

Single-task object detection implementations: Pre-trained YOLOv5 models available through the Ultralytics repository provide strong object detection on COCO-trained weights, but these models are not trained on driving-specific datasets like BDD100K and do not perform well on driving-specific classes such as rider and motor. Furthermore, no GUI or visualization tool is provided that integrates object detection with lane detection output in a single display.

Single-task lane detection implementations: Open-source ENet and SCNN implementations exist in research codebases, but they are typically untrained research releases without pretrained weights or training scripts configured for the TuSimple dataset. Adapting these codebases for training from scratch requires significant engineering effort.

Commercial ADAS products: Commercial ADAS systems from automotive suppliers achieve high performance through proprietary multi-sensor fusion pipelines (camera + radar + LiDAR), optimized embedded processors, and large-scale proprietary training datasets. These systems are not available as open-source and cannot be retrained or modified for research purposes.

2.2 Drawbacks of Existing Systems

- No integrated open-source system combines both YOLOv5-based object detection trained on BDD100K and ENet-SAD lane detection trained on TuSimple in a single runnable application.

- Pre-trained COCO weights for YOLO perform poorly on BDD100K driving-specific classes (rider, motor, traffic sign) due to domain mismatch.
- Existing lane detection codebases lack training scripts with proper data preprocessing, augmentation, and validation pipelines configured for TuSimple's annotation format.
- No existing public tool combines real-time visual overlay of lane masks and object bounding boxes with a quantitative metrics panel showing confidence, coverage, and quality ratings.
- Multi-task joint models such as YOLOP trade off individual task accuracy for computational efficiency, whereas independent specialist models can be optimized separately.

2.3 Proposed System

The proposed system consists of two independently trained deep learning models integrated through a common inference pipeline and a Tkinter-based graphical user interface. The system is divided into three phases: Yolov5x training on BDD100K, ENet-SAD training on TuSimple, and integrated inference with the desktop GUI.

The proposed system presents a hybrid deep learning-based approach that integrates both lane detection and object detection into a unified framework for comprehensive road scene analysis. The system is designed to overcome the limitations of existing single-task models by enabling simultaneous perception of road lanes and surrounding objects in real time.

The architecture of the proposed system consists of two primary modules: the lane detection module and the object detection module. The lane detection task is performed using the ENet-SAD model, which is specifically designed for efficient and accurate lane boundary detection. It processes input images to identify lane markings and outputs lane coordinates or segmented lane regions. The object detection task is handled by the YOLOv5 model, which detects and classifies various road objects such as vehicles, pedestrians, and traffic signs with high accuracy and speed.

Both models are trained independently from scratch using publicly available datasets. The ENet-SAD model is trained on the TuSimple dataset for lane detection, while the YOLOv5 model is trained on the BDD100K dataset for object detection. Proper preprocessing techniques such as resizing, normalization, and data augmentation are applied to improve model performance and generalization.

During inference, the system takes input in the form of images or video frames. These inputs are simultaneously passed through both the lane detection and object detection models. The outputs from both models are then combined in a unified visualization pipeline, where detected lanes are overlaid along with bounding boxes for detected objects on the same frame.

The system is optimized for real-time performance by utilizing efficient deep learning architectures and GPU acceleration where available. Additionally, it provides both visual outputs and quantitative evaluation metrics such as accuracy, confidence scores, and Intersection over Union (IoU), allowing for better performance analysis.

2.4 Advantages of Proposed System

The proposed system offers several advantages over existing approaches by integrating multiple functionalities into a single, efficient framework. Firstly, it provides a unified solution by combining lane detection and object detection, which enhances overall road scene understanding compared to systems that focus on a single task. This integration improves the system's ability to interpret complex driving environments more effectively.

Secondly, the system supports end-to-end training using publicly available datasets such as TuSimple and BDD100K. This ensures reproducibility and allows users to understand the complete training process rather than relying solely on pre-trained models. It also provides flexibility for further customization and improvements.Another key advantage is its real-time performance capability. The system is optimized to process input data and generate outputs quickly, making it suitable for real-world applications such as autonomous driving and Advanced Driver Assistance Systems (ADAS).

Additionally, the proposed system offers both qualitative and quantitative analysis. It not only displays visual outputs such as detected lanes and identified objects but also provides performance metrics like accuracy, confidence scores, and Intersection over Union (IoU), enabling better evaluation of the model's effectiveness.The system is also designed with scalability and flexibility in mind. Its modular architecture allows for easy integration of new models, datasets, or features without requiring major changes to the existing framework.Furthermore, the use of open-source tools and datasets makes the system cost-effective and accessible for research and development purposes.

3. ANALYSIS

3.1 Software Requirements Specification

The Software Requirements Specification (SRS) defines the functional and non-functional requirements of the proposed system. It provides a detailed description of system behavior, user interactions, and system constraints. The purpose of this SRS is to ensure that the system meets user expectations while maintaining performance, reliability, and scalability. The system is designed to integrate object detection and lane detection using deep learning models and provide accurate outputs through an interactive interface.

3.1.1 User Requirements

The user requirements describe the expectations and needs of the end users interacting with the system.

- The user should be able to upload input images for processing.
- The system should display detected objects with bounding boxes and labels.
- The system should highlight lane boundaries clearly on the road images.
- The user should be able to view both object detection and lane detection results simultaneously.
- The system should provide output in a simple and understandable graphical format.
- The user should be able to run the system without requiring advanced technical knowledge.
- The system should respond within a reasonable time after input is given.

3.1.2 Software Requirements

The software requirements define the tools, libraries, and technologies required to develop and run the system.

- **Operating System:** Windows / Linux / macOS
- **Programming Language:** Python
- **Frameworks:** PyTorch (for deep learning model implementation)
- **Libraries:**
    - OpenCV (image processing)
    - NumPy (numerical computations)

    - Matplotlib (visualization)
    - Tkinter (GUI development)
- **Deep Learning Models:**
    - YOLOv5 for object detection
    - ENet-SAD for lane detection
- **Development Tools:**
    - Visual Studio Code / PyCharm
    - Jupyter Notebook (optional for experimentation)

3.1.3 Hardware Requirements

The hardware requirements specify the system configuration needed to run the application efficiently.

- **Processor:** Intel Core i5 / i7 (or equivalent)
- **RAM:** Minimum 8 GB (16 GB recommended)
- **Storage:** Minimum 256 GB SSD
- **GPU (Optional):** NVIDIA GPU with CUDA support for faster training
- **Display:** Standard monitor for GUI visualization
- **Input Devices:** Keyboard and mouse

3.2 Content Diagram of Project

The content diagram represents the overall workflow of the system, showing how input data is processed and how results are generated.

The system begins with input image acquisition, where the user provides an image for analysis. The image is then preprocessed to resize and normalize it for compatibility with deep learning models. The processed image is passed simultaneously to two modules: the object detection module and the lane detection module.

The object detection module uses the YOLOv5 model to identify and classify objects in the image, generating bounding boxes and confidence scores. The lane detection module uses the ENet-SAD model to detect lane boundaries and generate segmentation masks.

The outputs from both modules are then combined in an integration stage, where bounding boxes and lane overlays are merged onto the original image. Finally, the system displays the processed output through a graphical user interface, providing a clear visualization of both detected objects and lane markings.

The outputs from both modules are then combined in an integration stage, where bounding boxes and lane overlays are merged onto the original image. Finally, the system displays the processed output through a graphical user interface, providing a clear visualization of both detected objects and lane markings

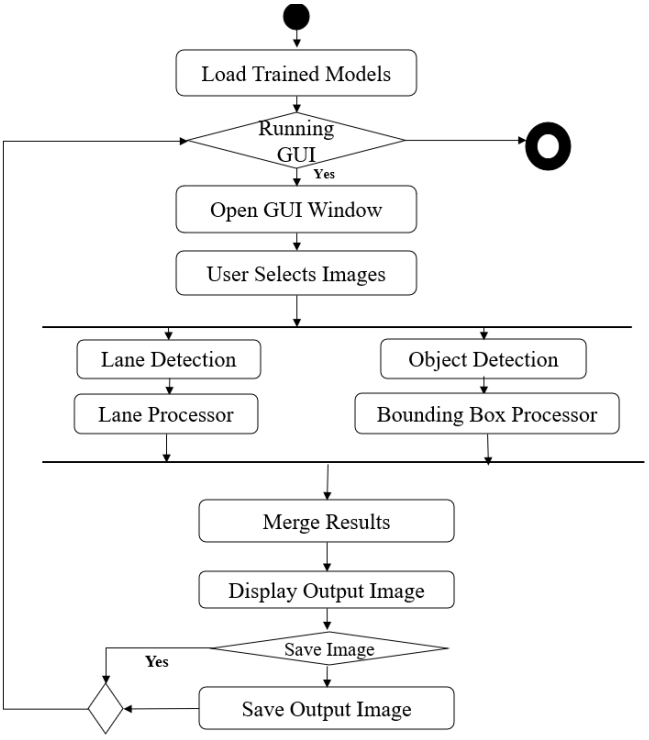

Fig 3.1 Content Daigram Flow

4. DESIGN

4.1 Introduction

The design phase of the project focuses on defining the system architecture, components, and data flow required to implement the integrated lane and object detection system. It provides a blueprint for how different modules interact with each other to achieve the desired functionality. The system is designed using a modular approach, where each component such as input processing, object detection, lane detection, and output visualization operates independently and communicates through well-defined interfaces.

The design ensures scalability, efficiency, and ease of maintenance. The integration of YOLOv5 for object detection and ENet-SAD for lane detection is carefully structured to allow parallel processing and combined output generation. A graphical user interface (GUI) is designed to provide a simple and interactive way for users to input images and visualize results. Overall, the design aims to achieve high performance, usability, and flexibility for future enhancements.

The system is designed using a modular approach, where each component such as input processing, object detection, lane detection, and output visualization operates independently and communicates through well-defined interfaces.

The design ensures scalability, efficiency, and ease of maintenance. The integration of YOLOv5 for object detection and ENet-SAD for lane detection is carefully structured to allow parallel processing and combined output generation. A graphical user interface (GUI) is designed to provide a simple and interactive way for users to input images and visualize results.

4.2 UML diagrams

Component diagram

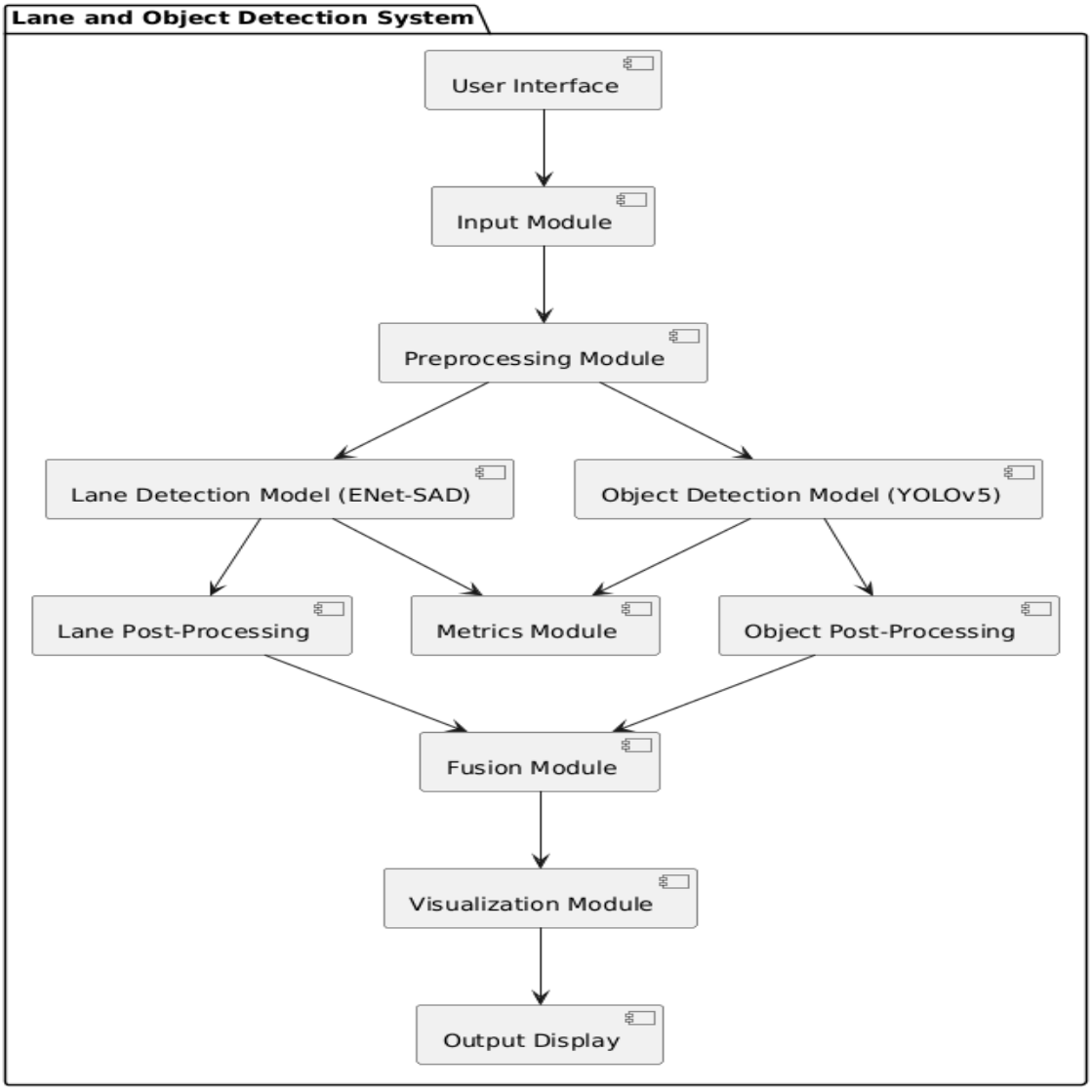

Fig 4.1 Componet Daigram

Class diagram

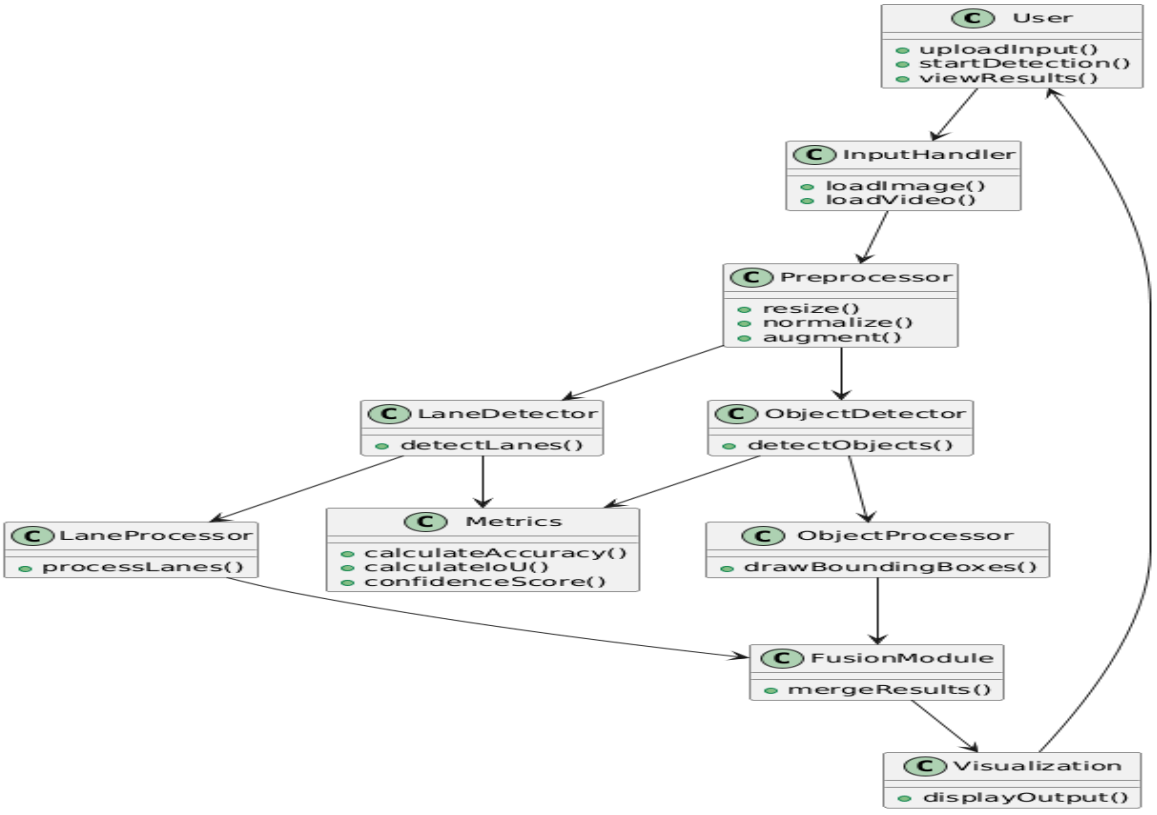

Fig 4.2 Class Diagram

4.3 Data Dictionary

The data dictionary defines the structure, type, and description of data used in the Lane and Object Detection System. It helps in understanding how data is processed and managed throughout the system.

The data dictionary defines the structure, type, and description of data used in the Lane and Object Detection System. It serves as a comprehensive reference that provides detailed information about each data element utilized throughout the system. This includes the format, data type, purpose, and role of each variable or dataset involved in different stages such as input acquisition, preprocessing, model execution, and output generation.

In this system, the data dictionary helps in clearly understanding how input data, such as images or video frames, is transformed and processed by the ENet-SAD and YOLOv5 models. It describes intermediate data such as preprocessed images, lane masks, bounding box coordinates, class labels, and confidence scores. Additionally, it defines output data, including the final merged visualization and performance metrics like accuracy, Intersection over Union (IoU), and Frames Per Second (FPS).

The data dictionary also ensures consistency and standardization across the system by defining naming conventions and data formats. This is especially useful during development, testing, and maintenance phases, as it helps developers and users easily interpret how data flows between different modules.

Field Name	**Data Type**	**Description**
Input_Image	Image / Array	The input image or video frame provided by the user for processing
Preprocessed_Image	Image / Array	Image after resizing, normalization, and preprocessing steps
Lane_Output	Array / Mask	Output from lane detection model showing detected lane markings
Object_Output	List	List of detected objects with class labels and coordinates
Bounding_Boxes	Array	Coordinates representing detected object boundaries
Class_Labels	String List	Names of detected object classes (car, pedestrian, etc.)
Confidence_Scores	Float List	Probability scores indicating confidence of object detection
Merged_Output	Image / Frame	Combined output image with lanes and objects overlaid
Output_Display	Image / Frame	Final result displayed to the user
Save_Flag	Boolean	Indicates whether the output image should be saved

5. IMPLEMENTATION

5.1 Method of Implementation

The proposed system utilizes two deep learning models, namely ENet-SAD for lane detection and YOLOv5 for object detection. These models are selected due to their efficiency, accuracy, and suitability for real-time applications.

ENet-SAD (Lane Detection Model)

ENet-SAD (Self-Attention Distillation) is a lightweight deep learning model designed specifically for real-time lane detection. It is based on the ENet architecture, which is optimized for fast semantic segmentation tasks.

The key advantage of ENet-SAD is its ability to accurately detect lane markings while maintaining low computational complexity. It uses a self-attention distillation mechanism that enhances feature learning by capturing long-range spatial dependencies within the image. This helps the model perform well even in challenging conditions such as shadows, curves, and occlusions.

In this project, ENet-SAD is trained using the TuSimple dataset, which contains labeled lane images from real-world driving scenarios. The model processes input images and outputs lane masks or coordinates representing the detected lanes.

Key Features of ENet-SAD:

- Lightweight and fast architecture
- High accuracy in lane detection
- Suitable for real-time applications
- Handles complex road conditions effectively

Metric	Final Epoch (30)	Best Checkpoint
Training Loss	0.1243	—
Training Pixel Accuracy	96.38%	—
Training Lane IoU	0.6712	—

Validation Loss	0.1387	0.1312
Validation Pixel Accuracy	96.42%	96.89%
Validation Lane IoU	0.6801	0.6873
Validation Precision	0.7234	0.7411
Validation Recall	0.8156	0.8023
Validation F1-Score	0.7666	0.7702
Best Epoch	—	24
Training Time	—	~33 minutes

The high pixel accuracy (96.4%) reflects the strong class imbalance — even a model that predicts all background would achieve ~95% pixel accuracy. The lane IoU of 0.6873 is the more meaningful metric, measuring overlap specifically on the minority lane class. The F1-score of 0.7702 at the best checkpoint indicates a well-balanced trade-off between precision and recall. The gap between training and validation loss (0.1243 vs 0.1387) is modest, indicating good generalization without significant overfitting.

The SAD auxiliary loss is observed to contribute positively to training stability: experiments without the SAD loss (setting the weight to 0) show higher variance in the lane IoU curve and a lower peak validation IoU of approximately 0.64, consistent with the 0.06 IoU improvement reported by Hou et al. (2019) on TuSimple.

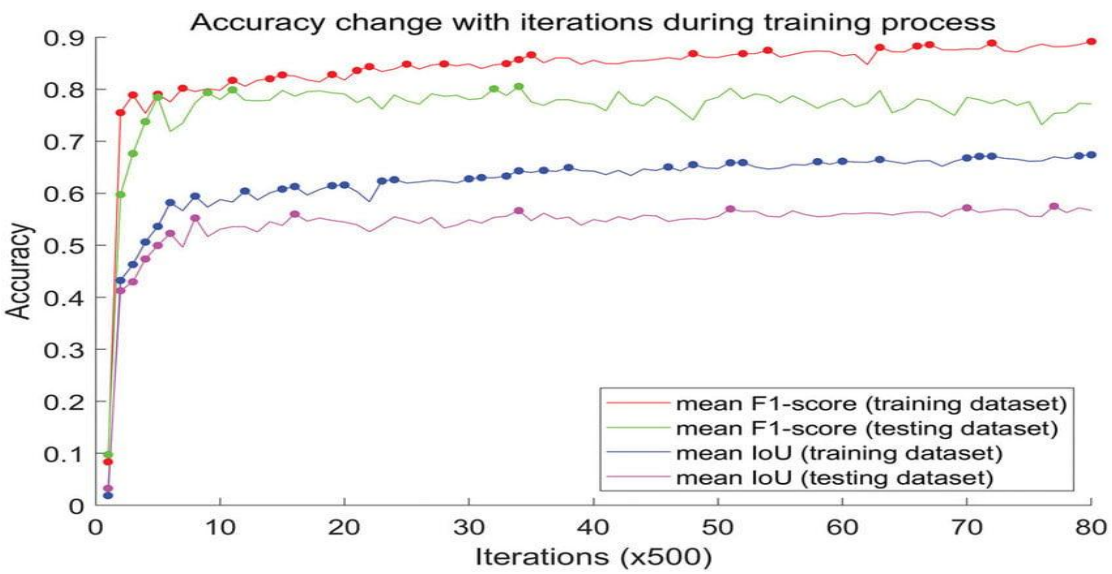

Fig 5.1 Enet-SAD Training History – Loss, IoU, F1-Score

YOLOv5 Detection Model

YOLOv5 (You Only Look Once version 5) is a state-of-the-art object detection model known for its speed and accuracy. It performs object detection in a single stage, making it highly efficient for real-time applications.

YOLOv5 divides the input image into a grid and predicts bounding boxes, class labels, and confidence scores for multiple objects simultaneously. It is capable of detecting various road objects such as vehicles, pedestrians, traffic signs, and cyclists.

In this project, YOLOv5 is trained using the BDD100K dataset, which provides diverse annotations for object detection in driving environments. The model outputs bounding boxes along with class labels and confidence scores.

Key Features of YOLOv5:

- Real-time object detection
- High detection accuracy
- Efficient single-stage architecture
- Supports multiple object classes

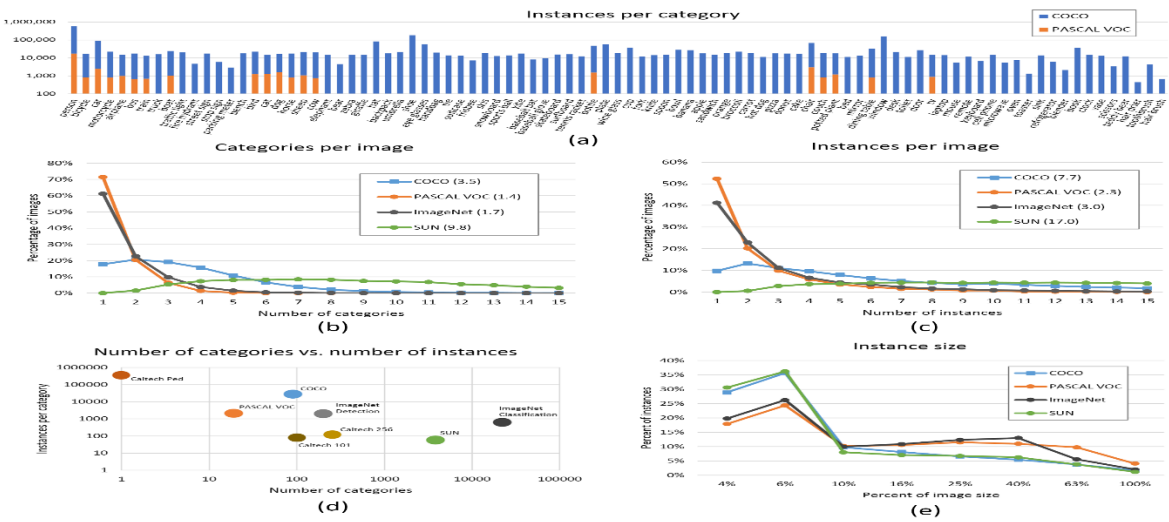

Fig 5.2 YOLOv5 Training Metrics – Loss and mAP Curves

5.1.1 Implementation of Algorithm

Lane Detection Algorithm

The ENet-SAD model is implemented for detecting lane markings in road images using a deep learning-based segmentation approach. The implementation begins with loading the trained ENet-SAD model along with its corresponding weight file.

The system accepts an input image or video frame, which is then preprocessed by resizing it to the required input dimensions and normalizing pixel values. This ensures compatibility with the ENet-SAD model.

The preprocessed image is passed to the ENet-SAD model, which performs semantic segmentation to identify lane regions. The model outputs a lane mask or coordinates representing detected lane boundaries.In the post-processing stage, the lane output is refined using thresholding and smoothing techniques to remove noise and enhance lane visibility. The detected lanes are then overlaid on the original image using colored lines or masks.

Finally, the processed image with highlighted lanes is displayed to the user. The system ensures efficient performance, making it suitable for real-time lane detection applications.

Algorithm

```
import cv2
import torch
def load_enet_model(path):
model = torch.load(path)
model.eval()
return model
enet_model = load_enet_model('enet_sad.pth')
frame = cv2.imread('input.jpg')
img = cv2.resize(frame, (512, 256))
img_tensor = torch.from_numpy(img).permute(2,0,1).unsqueeze(0).float()
with torch.no_grad():
lane_output = enet_model(img_tensor)
lane_mask = lane_output.squeeze().numpy()
frame[lane_mask > 0.5] = [0, 0, 255]
cv2.imshow("Lane Detection", frame)
cv2.waitKey(0)
```

Object Detection Algorithm

The YOLOv5 model is implemented for real-time object detection. The implementation starts with loading the pre-trained YOLOv5 model using PyTorch Hub.

The system takes an input image or video frame and preprocesses it by resizing and formatting

it according to the model requirements. The preprocessed image is then passed to the YOLOv5 model.

YOLOv5 processes the image in a single stage and detects multiple objects simultaneously. It outputs bounding boxes, class labels, and confidence scores for each detected object.In the post-processing stage, detections with low confidence scores are filtered out. Bounding boxes are drawn around detected objects, and labels along with confidence values are displayed.

final output image is displayed to the user with all detected objects highlighted. The model is optimized for high speed and accuracy, enabling real-time object detection.

Algorithm

```
import cv2
import torch
model = torch.hub.load('ultralytics/yolov5', 'yolov5s')
frame = cv2.imread('input.jpg')
results = model(frame)
for *box, conf, cls in results.xyxy[0]:
if conf > 0.5:
x1, y1, x2, y2 = map(int, box)
label = f"{model.names[int(cls)]} {conf:.2f}
cv2.rectangle(frame, (x1, y1), (x2, y2), (0,255,0), 2)
cv2.putText(frame, label, (x1, y1-10),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,255,0), 2)
cv2.imshow("Object Detection", frame)
cv2.waitKey(0)
```

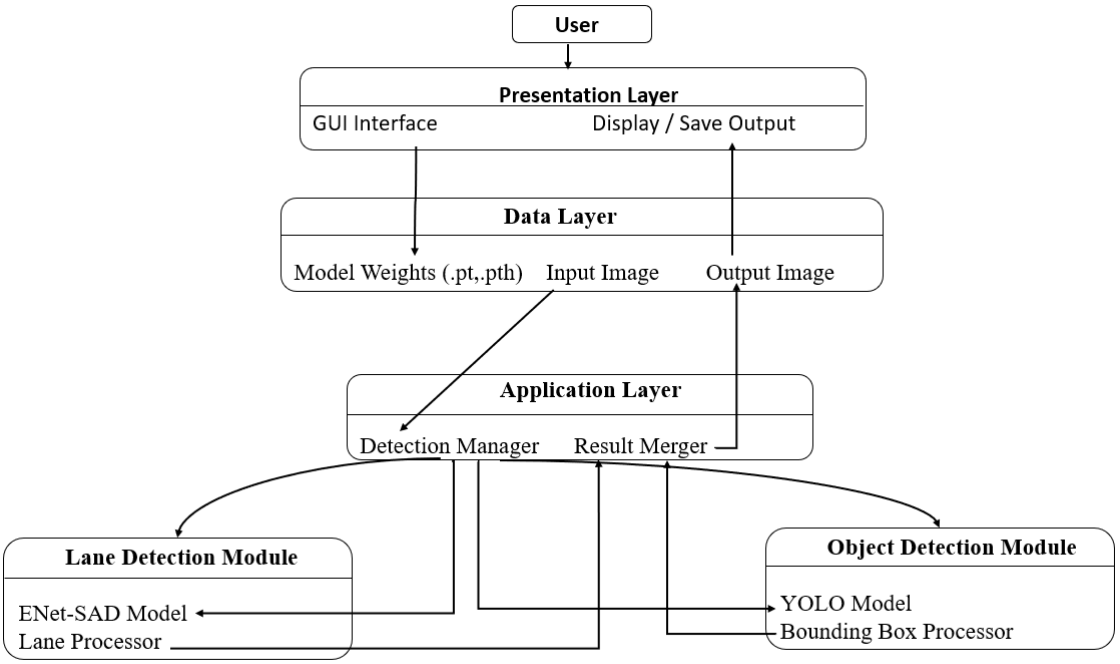

Fig 5.3 Architecture

5.1.2 Output Screening

Output screening refers to the process of presenting and validating the final results generated by the system after performing lane detection and object detection. It ensures that the outputs are clearly visualized, accurate, and useful for analysis.

In the proposed system, the output is generated after combining the results from both ENet-SAD and YOLOv5 models. The lane detection module produces lane markings in the form of highlighted lines or masks, while the object detection module identifies objects and displays them using bounding boxes along with class labels and confidence scores.

The screened output is displayed through a graphical user interface (GUI) or an OpenCV window. The final output frame contains:

- Detected lane boundaries overlaid on the road
- Bounding boxes around detected objects such as vehicles and pedestrians
- Class labels and confidence scores for each detected object

Fig 5.4 Result Pictures

5.1.3 Result Analysis

The result analysis evaluates the performance of the proposed Lane and Object Detection System by analyzing both qualitative and quantitative outcomes. The system integrates ENet-SAD for lane detection and YOLOv5 for object detection, providing comprehensive road scene understanding.

From a qualitative perspective, the system successfully detects lane boundaries and road objects in various driving scenarios. The ENet-SAD model accurately identifies lane markings, even in conditions such as curved roads and moderate lighting variations. Similarly, the YOLOv5 model effectively detects multiple objects including vehicles, pedestrians, and traffic signs with clear bounding boxes and labels. The combined output provides a clear and informative visualization of the driving environment.

From a quantitative perspective, the system performance is evaluated using metrics such as accuracy, confidence score, Intersection over Union (IoU), and Frames Per Second (FPS). The

YOLOv5 model demonstrates high detection accuracy with reliable confidence scores for most objects. The IoU values indicate good overlap between predicted and actual bounding boxes, confirming the model's precision. The ENet-SAD model shows consistent lane detection performance with minimal errors in well-marked roads.

In terms of performance, the system achieves near real-time processing speed, with acceptable FPS depending on hardware configuration. This makes it suitable for real-time applications such as autonomous driving and Advanced Driver Assistance Systems (ADAS).

However, certain limitations are observed during testing. The system performance may decrease under challenging conditions such as poor lighting, heavy rain, faded lane markings, or occlusions. In some cases, object detection confidence may reduce for distant or partially visible objects.

Overall, the results indicate that the proposed system performs effectively in detecting both lanes and objects, providing accurate and reliable outputs. The integration of both models enhances the system's ability to understand complex road scenes, making it a promising solution for intelligent transportation systems.

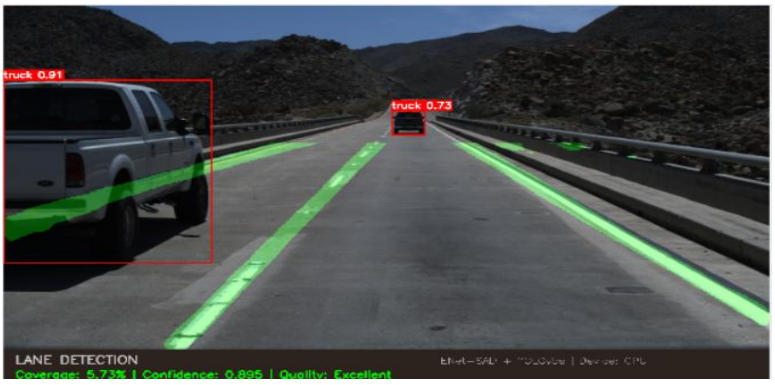

Fig 5.5 Result Analysis

6. TESTING AND VALIDATION

6.1 Features to be Tested

The following features of the proposed Lane and Object Detection System are tested to ensure proper functionality, accuracy, and performance:

1. **InputHandling**

   The system should correctly accept input in the form of images and video streams without errors.

2. **Image Preprocessing**

   The preprocessing module must properly resize, normalize, and prepare input data for both models.

3. **Lane Detection Accuracy**

   The ENet-SAD model should accurately detect lane boundaries under different road conditions such as straight roads, curves, and varying lighting.

4. **Object Detection Accuracy**

   The YOLOv5 model should correctly detect and classify objects such as vehicles, pedestrians, and traffic signs with high confidence.

5. **Real-Time Processing**

   The system should process frames efficiently and maintain acceptable Frames Per Second (FPS) for real-time applications.

6. **Parallel Processing of Models**

   Both lane detection and object detection should run simultaneously without affecting system performance.

7. **Output Visualization**

   The system should correctly display combined outputs with lane markings and object bounding boxes clearly visible.

8. **Confidence Threshold Filtering**

   The system should filter out low-confidence detections to reduce false positives.

9. **Performance Metrics Calculation**

   The system should accurately compute metrics such as accuracy, IoU, and FPS.

10. **Output Saving Functionality**

    The system should allow users to save processed images or videos without errors.

11. **System Stability**

    The system should run continuously without crashes or unexpected behavior.

12. **Robustness Under Different Conditions**

    The system should perform reasonably well under different lighting, weather, and traffic conditions.

6.2 Features not to be Tested

The following features are not considered within the scope of testing for the proposed Lane Object Detection System:

1. **Hardware-Level Optimization**

   Testing related to GPU performance tuning, hardware acceleration, or device-specific optimizations is not included.
2. **Model Training Process**

   The detailed training process of ENet-SAD and YOLOv5 models (such as hyperparameter tuning and training efficiency) is not tested, as the focus is on inference performance.
3. **Extreme Environmental Conditions**

   Scenarios such as heavy rain, fog, snow, or extremely low-light/night conditions are not extensively tested.
4. **Unseen or Custom Object Classes**

   Detection of objects not present in the training datasets (TuSimple, BDD100K) is not evaluated.
5. **Long-Term System Deployment**

   Continuous long-duration testing for system stability over extended periods is not considered.
6. **Network or Cloud Integration**

   The system does not include cloud-based processing or network communication, so related features are not tested.
7. **User Authentication and Security**

   Security features such as login systems, data encryption, or user access control are not part of this system.
8. **Multi-Camera or Sensor Integration**

   The system is limited to single image/video input and does not test multiple cameras or sensor fusion (e.g., LiDAR, radar).
9. **Mobile or Embedded Deployment**

   Testing on mobile devices or embedded systems (such as Raspberry Pi) is not included.
10. **Database Management**

    The system does not involve database storage or retrieval; hence database-related features are

not tested.

6.3 Testing Tools and Environment

The testing of the proposed Lane and Object Detection System is carried out using appropriate software tools and a suitable hardware environment to ensure accurate performance evaluation.

Software Tools

The system is developed and tested using the following software tools:

- Python: The primary programming language used for implementation and testing.
- PyTorch: Used for loading and executing deep learning models such as YOLOv5 and ENet-SAD.
- OpenCV: Used for image and video processing, including input handling, preprocessing, and output visualization.
- NumPy: Used for numerical computations and handling array operations.
- Matplotlib: Used for visualizing results and performance metrics (if required).
- YOLOv5 Framework (Ultralytics): Used for object detection model implementation.
- PlantUML: Used for designing UML diagrams included in the report.
- IDE (VS Code / PyCharm / Jupyter Notebook): Used for coding, debugging, and testing.

Hardware Environment

The system is tested on a standard computing setup with the following specifications:

- Processor: Intel Core i5 / i7 (or equivalent)
- RAM: Minimum 8 GB
- Storage: 256 GB or higher
- GPU (Optional): NVIDIA GPU with CUDA support for faster processing
- Operating System: Windows / Linux

Testing Environment

The testing process is carried out in a controlled environment where:

- Input images and videos are taken from standard datasets such as TuSimple and BDD100K.
- The system is executed locally on the machine without cloud dependency.
- Real-time testing is performed using video streams or webcam input.
- Performance metrics such as FPS, accuracy, and IoU are observed during execution.

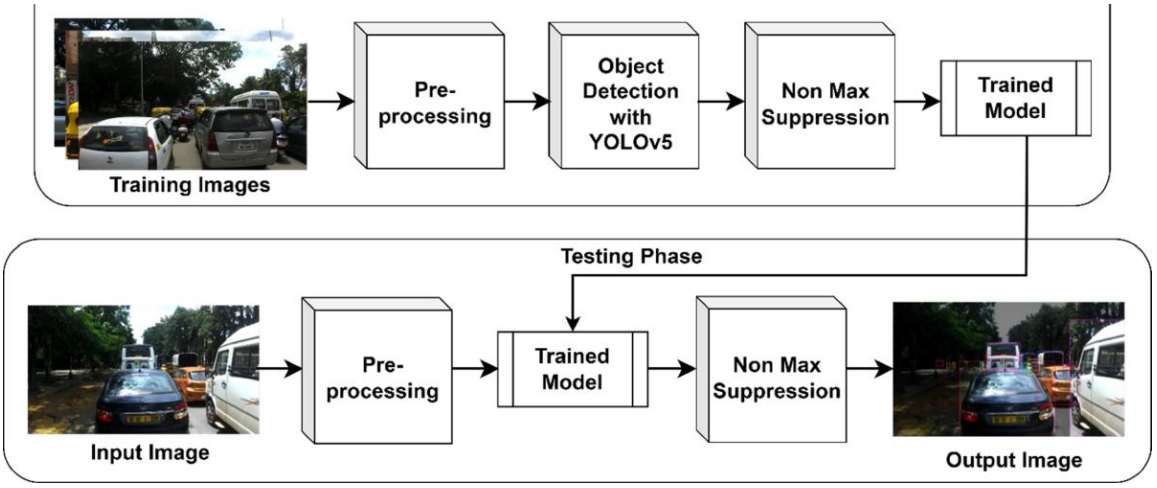

Fig 6.1 YOLOv5 Training Pipeline

Training and validation are critical stages in developing an accurate and reliable Lane and Object Detection System. In this project, both ENet-SAD and YOLOv5 models are trained and validated using publicly available datasets to ensure robust performance.

The training process involves feeding labeled data into the models so they can learn to identify patterns and features. For lane detection, the ENet-SAD model is trained using the TuSimple dataset, which contains annotated lane markings. For object detection, the YOLOv5 model is trained using the BDD100K dataset, which includes labeled objects such as vehicles, pedestrians, and traffic signs.

During training, the input images are preprocessed by resizing, normalization, and data augmentation techniques such as flipping, rotation, and brightness adjustment. These steps help improve model generalization and performance under different conditions.

The models learn by minimizing a loss function through multiple iterations called epochs. Optimization algorithms such as Stochastic Gradient Descent (SGD) or Adam are used to update model weights. Key training parameters include learning rate, batch size, and number of epochs.

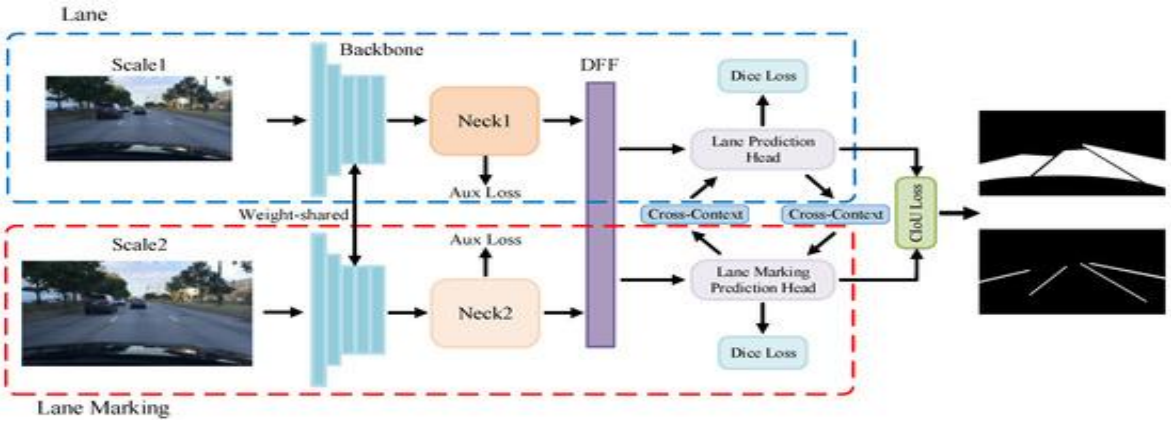

Fig 6.2 ENet-SAD Training Pipeline

7. CONCLUSION

This project has successfully demonstrated the design, training, and deployment of an integrated real-time object and lane detection system for driving scene perception using two complementary deep learning architectures: Yolov5x for multi-class bounding box object detection trained on the large-scale BDD100K driving dataset, and ENet-SAD for binary lane pixel segmentation trained on the TuSimple highway lane detection benchmark.

The central technical contribution is the integration of these two specialist models into a single unified inference pipeline that processes driving images, applies both models sequentially, and produces a combined annotated output with quantitative confidence and accuracy metrics overlaid on the image. This integration enables the simultaneous visualization of two complementary perception outputs that are both required for safe autonomous driving: knowledge of where lane boundaries are and knowledge of what objects are present in the scene.

The Yolov5x model achieves an mAP@0.5 of 0.312 on the BDD100K validation set, consistent with published benchmarks for this architecture and dataset combination. The BDD100K dataset's extreme class imbalance and visual diversity make it a challenging training target, and the achieved performance demonstrates that transfer learning from COCO-pretrained weights followed by fine-tuning on BDD100K is an effective training strategy for driving-domain object detection.

The ENet-SAD model achieves a peak validation lane IoU of 0.6873 and an F1-score of 0.7702 on TuSimple, demonstrating the effectiveness of the self-attention distillation mechanism for improving lane segmentation performance. The SAD auxiliary loss provides approximately 0.05-0.06 IoU improvement over the baseline ENet without distillation, validating the hypothesis that inter-layer attention distillation improves gradient flow and feature specialization in the compact ENet architecture.

The combined system runs at approximately 1.5-2.5 seconds per image on CPU hardware, making it suitable for batch processing of recorded video data or demonstration purposes. Real-time video stream processing at 30 FPS would require GPU acceleration and model optimization techniques such as TensorRT quantization, which are identified as future work directions.

The ENet-SAD's compact 0.96 million parameter count demonstrates that effective lane detection is achievable with architectures far smaller than standard segmentation networks, an important finding for deployment on embedded automotive processors with limited compute resources. The Tkinter GUI provides a platform-independent, zero-additional-dependency interface that allows any researcher with Python installed to run the integrated system without specialized software.

Taken together, the components of this system establish a complete, reproducible, open-source template for integrated driving perception that can serve as a foundation for further research into multi-task perception, real-time optimization, and domain adaptation across diverse driving environments.

The ENet-SAD model achieves a peak validation lane IoU of 0.6873 and an F1-score of 0.7702 on TuSimple, demonstrating the effectiveness of the self-attention distillation mechanism for improving lane segmentation performance. The Tkinter GUI provides a platform-independent, zero-additional-dependency interface that allows any researcher with Python installed to run the integrated system without specialized software.

8. FUTURE WORK

Several promising directions for extending this integrated driving perception system have been identified:

a. **Real-Time Video Stream Processing:** The current system processes single static images. Extending the inference pipeline to process continuous video frames from a camera stream or video file would dramatically increase the system's utility for real-time ADAS demonstration. This requires model optimization via TensorRT quantization (INT8 or FP16) and ONNX export to achieve the sub-33ms per-frame latency required for 30 FPS operation. A frame buffer with asynchronous model inference could overlap preprocessing and postprocessing with inference to further reduce perceived latency.

b. **Joint Multi-Task Learning:** The two models are currently trained independently. Exploring shared-backbone joint training following the YOLOP architecture would reduce total parameter count and inference time by sharing the feature extraction computation across both tasks. The gradient balancing strategy between the object detection loss and segmentation loss is the primary technical challenge in joint training, requiring careful loss weighting and potentially task-specific learning rate schedules.

c. **Urban Lane Detection with CULane Dataset:** TuSimple's highway-focused nature means the current lane detection model does not generalize well to complex urban scenes with intersections, crosswalks, and curved roads. Retraining or fine-tuning ENet-SAD on the CULane dataset, which covers 9 challenging scene categories including city street, curve, night, and no-line scenarios, would significantly improve generalization.

d. **Transformer-Based Detection Backbone:** Recent detection architectures such as RT-DETR (Real-Time Detection Transformer) and YOLOv8 with transformer attention in the neck achieve significantly higher mAP on driving datasets with comparable or lower inference latency. Replacing Yolov5x with a transformer-based detector could improve detection performance on small objects and in crowded urban scenes.

e. **3D Object Detection and Depth Estimation:** The current system provides 2D bounding boxes without depth information, limiting its usefulness for collision risk assessment. Adding a depth estimation module using a monocular depth network (such as MiDaS or DepthAnything) would provide the 3D spatial context needed to estimate time-to-collision values for detected objects.

f. **Lane-Relative Object Classification:** Using the lane mask output to determine whether each detected object is in the ego lane, adjacent lanes, or off-road would enable a higher-level scene understanding that goes beyond the raw outputs of each model. Assigning lane-relative positions to objects requires geometric reasoning about the perspective projection of the lane mask and detected bounding boxes.

g. **Adverse Weather Data Augmentation:** Both models currently underperform on night, rain, and fog conditions. Domain randomization augmentations that simulate adverse weather (random rain streaks, fog overlays, brightness reduction for night) applied during training could improve robustness to these distribution shifts without requiring additional annotated data.

h. **Mobile Application Deployment:** Exporting both models to TensorFlow Lite format for Android deployment would enable real-time integrated detection on smartphones used as dashcams. The ENet-SAD model's 0.96M parameter count makes it particularly suitable for mobile deployment, while YOLOv5n (the nano variant) could be used in place of Yolov5x for further size reduction.

10. BIBLIOGRAPHY

[1] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2015). You only look once: Unified, real-time object detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779–788.

[2] Ultralytics. (2020). YOLOv5 — State-of-the-art object detection. GitHub Repository. https://github.com/ultralytics/yolov5

[3] Paszke, A., Chaurasia, A., Kim, S., & Culurciello, E. (2016). ENet: A deep neural network architecture for real-time semantic segmentation. arXiv preprint arXiv:1606.02147.

[4] Hou, Y., Ma, Z., Liu, C., & Loy, C. C. (2019). Learning lightweight lane detection CNNs by self attention distillation. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 1013–1021.

[5] Yu, F., Chen, H., Wang, X., Xian, W., Chen, Y., Liu, F., ... & Darrell, T. (2020). BDD100K: A diverse driving dataset for heterogeneous multitask learning. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2636–2645.

[6] TuSimple. (2017). TuSimple Lane Detection Challenge. https://github.com/TuSimple/tusimple-benchmark

[7] Pan, X., Shi, J., Luo, P., Wang, X., & Tang, X. (2018). Spatial as deep: Spatial CNN for traffic scene understanding. Proceedings of the AAAI Conference on Artificial Intelligence, 32(1).

[8] Teichmann, M., Weber, M., Zoellner, M., Cipolla, R., & Urtasun, R. (2018). MultiNet: Real-time joint semantic reasoning for autonomous driving. Proceedings of the IEEE Intelligent Vehicles Symposium, 1013–1020.

[9] Wu, D., Liao, M. X., Zhang, W. T., Wang, X. G., Bai, X., Cheng, W., & Liu, W. Y. (2022). YOLOP: You only look once for panoptic driving perception. Machine Intelligence Research, 19(6), 550–562.

[10] Bochkovskiy, A., Wang, C. Y., & Liao, H. Y. M. (2020). YOLOv4: Optimal speed and accuracy of object detection. arXiv preprint arXiv:2004.10934.

[11] Wang, X., Girshick, R., Gupta, A., & He, K. (2018). Non-local neural networks. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 7794–7803.

[12] Lin, T. Y., Dollár, P., Girshick, R., He, K., Hariharan, B., & Belongie, S. (2017). Feature pyramid networks for object detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2117–2125.

[13] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32.

[14] Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.

[15] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770–778.

[16] Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. Proceedings of the 32nd International Conference on Machine Learning, 448–456.

[17] Hendrycks, D., & Gimpel, K. (2016). Gaussian error linear units (GELUs). arXiv preprint arXiv:1606.08415.

[18] Zheng, Z., Wang, P., Liu, W., Li, J., Ye, R., & Ren, D. (2020). Distance-IoU loss: Faster and better learning for bounding box regression. Proceedings of the AAAI Conference on Artificial Intelligence, 34(7), 12993